

6주차 생성형 ai 이용

1. 프롬프트

claude에게
로보로보 ai자율주행 자동차 키트 샀는데 기본적인 바퀴 동작들이 되는지 확인할 수 있는 테스트 코드 만들어줘

2. 결과 코드

```
// 로보로보 AI 자율주행 자동차 기본 동작 테스트 코드

// 모터 핀 설정 (실제 키트의 핀 번호에 맞게 수정하세요)
#define LEFT_MOTOR_PIN1 5
#define LEFT_MOTOR_PIN2 6
#define RIGHT_MOTOR_PIN1 9
#define RIGHT_MOTOR_PIN2 10

// 모터 속도 설정 (0~255)
int motorSpeed = 150;

void setup() {
  // 시리얼 통신 초기화
  Serial.begin(9600);
  Serial.println("로보로보 자동차 테스트 시작!");
  Serial.println("=====");

  // 모터 핀을 출력으로 설정
  pinMode(LEFT_MOTOR_PIN1, OUTPUT);
  pinMode(LEFT_MOTOR_PIN2, OUTPUT);
  pinMode(RIGHT_MOTOR_PIN1, OUTPUT);
  pinMode(RIGHT_MOTOR_PIN2, OUTPUT);

  // 초기화 대기
  delay(2000);
}
```

```

}

void loop() {
  // 1. 전진 테스트
  Serial.println("1. 전진 테스트");
  moveForward();
  delay(2000);
  stopMotors();
  delay(1000);

  // 2. 후진 테스트
  Serial.println("2. 후진 테스트");
  moveBackward();
  delay(2000);
  stopMotors();
  delay(1000);

  // 3. 좌회전 테스트
  Serial.println("3. 좌회전 테스트");
  turnLeft();
  delay(1500);
  stopMotors();
  delay(1000);

  // 4. 우회전 테스트
  Serial.println("4. 우회전 테스트");
  turnRight();
  delay(1500);
  stopMotors();
  delay(1000);

  // 5. 제자리 좌회전 테스트
  Serial.println("5. 제자리 좌회전 테스트");
  spinLeft();
  delay(1500);
  stopMotors();
  delay(1000);
}

```

```

// 6. 제자리 우회전 테스트
Serial.println("6. 제자리 우회전 테스트");
spinRight();
delay(1500);
stopMotors();
delay(1000);

// 7. 속도 변화 테스트
Serial.println("7. 속도 변화 테스트");
testSpeedControl();
delay(1000);

Serial.println("=====");
Serial.println("모든 테스트 완료! 5초 후 재시작");
Serial.println("=====");
delay(5000);
}

// 전진
void moveForward() {
  analogWrite(LEFT_MOTOR_PIN1, motorSpeed);
  analogWrite(LEFT_MOTOR_PIN2, 0);
  analogWrite(RIGHT_MOTOR_PIN1, motorSpeed);
  analogWrite(RIGHT_MOTOR_PIN2, 0);
}

// 후진
void moveBackward() {
  analogWrite(LEFT_MOTOR_PIN1, 0);
  analogWrite(LEFT_MOTOR_PIN2, motorSpeed);
  analogWrite(RIGHT_MOTOR_PIN1, 0);
  analogWrite(RIGHT_MOTOR_PIN2, motorSpeed);
}

// 좌회전 (왼쪽 모터 느리게)
void turnLeft() {
  analogWrite(LEFT_MOTOR_PIN1, motorSpeed / 2);
  analogWrite(LEFT_MOTOR_PIN2, 0);

```

```

    analogWrite(RIGHT_MOTOR_PIN1, motorSpeed);
    analogWrite(RIGHT_MOTOR_PIN2, 0);
}

// 우회전 (오른쪽 모터 느리게)
void turnRight() {
    analogWrite(LEFT_MOTOR_PIN1, motorSpeed);
    analogWrite(LEFT_MOTOR_PIN2, 0);
    analogWrite(RIGHT_MOTOR_PIN1, motorSpeed / 2);
    analogWrite(RIGHT_MOTOR_PIN2, 0);
}

// 제자리 좌회전
void spinLeft() {
    analogWrite(LEFT_MOTOR_PIN1, 0);
    analogWrite(LEFT_MOTOR_PIN2, motorSpeed);
    analogWrite(RIGHT_MOTOR_PIN1, motorSpeed);
    analogWrite(RIGHT_MOTOR_PIN2, 0);
}

// 제자리 우회전
void spinRight() {
    analogWrite(LEFT_MOTOR_PIN1, motorSpeed);
    analogWrite(LEFT_MOTOR_PIN2, 0);
    analogWrite(RIGHT_MOTOR_PIN1, 0);
    analogWrite(RIGHT_MOTOR_PIN2, motorSpeed);
}

// 정지
void stopMotors() {
    analogWrite(LEFT_MOTOR_PIN1, 0);
    analogWrite(LEFT_MOTOR_PIN2, 0);
    analogWrite(RIGHT_MOTOR_PIN1, 0);
    analogWrite(RIGHT_MOTOR_PIN2, 0);
}

// 속도 변화 테스트
void testSpeedControl() {

```

```

Serial.println(" - 저속 전진");
motorSpeed = 100;
moveForward();
delay(1500);

Serial.println(" - 중속 전진");
motorSpeed = 150;
moveForward();
delay(1500);

Serial.println(" - 고속 전진");
motorSpeed = 200;
moveForward();
delay(1500);

motorSpeed = 150; // 기본 속도로 복원
stopMotors();
}

```

3. 문제점

- 역방향 전환 시 데드타임(Dead-time) 없음 → 쇼트/전류 스파이크 위험
 - 기존 코드는 **전진 → 후진** 같은 극성 전환을 곧바로 수행한다. H-브릿지에서 하이가드-로이가드가 겹치면 순간적으로 양쪽이 도통되어 쇼트/발열/리셋 위험이 있다.
 - **보완 방향 제시:** 방향을 바꾸기 직전에 **PWM=0**으로 멈춤 → 수십 ms 대기(데드타임) → 새 방향 적용 순서를 강제한다.
- 속도 변수 관리와 타입/범위 안전성 부족
 - **motorSpeed** 가 **int** 로 전역 변경되고, 함수 내부에서 값이 바뀌며 **클램프(constrain)**도 없다. 드라이버는 0~255 범위를 기대한다.
 - **보완 방향 제시:** 외부 API는 **부호 있는 속도(signed)**로 받되 내부에서 **abs** 와 **constrain(0~255)** 로 PWM을 계산한다. 전역 상태를 바꾸지 않고, 테스트 함수는 **지역 변수**로 속도를 바꿔 사용한다.
- 동작 함수가 중복/하드코딩됨 → 유지보수 어려움
 - **moveForward/Backward/turnLeft/...** 가 핀 쓰기를 반복한다. 바퀴 교체·배선 뒤바뀜·핀 변경 시 실수 유발.

- **보완 방향 제시:** `drive(leftSpeed, rightSpeed)` 하나로 추상화(왼/오른 바퀴에 부호 있는 속도 입력). `stopCoast()` (코스팅) / `stopBrake()` (능동 브레이크)도 분리해 제동 특성 제어.